Sairot

Scouting Days Tests

App Documentation

Version: 1.0.0+1
Generated: 2025-11-27 14:14:33

Table of Contents

1. Executive Summary

Sairot is a Flutter-based mobile application designed for managing and tracking scouting/reconnaissance day assessments and tests. The application serves instructors in organizing events, tracking participant performance across multiple test types, and generating comprehensive reports. Built with modern Flutter architecture and Firebase backend, the app provides offline-first functionality with cloud synchronization capabilities.

2. App Overview

2.1 Purpose

The application is designed for instructors to manage scouting day events, track participant performance across various physical and mental assessments, and generate comprehensive reports. The app provides a comprehensive workflow for instructors to manage events, track tests, and record participant grades.

2.2 User Roles

Instructors:

• Login with instructor ID
• Create and manage events
• Track test progress (Meshulash, Alonka, Bur, Sakim, Leadership, Interview)
• Record participant grades and comments
• View participant performance and status
• Generate event reports

2.3 Key Features

• **Event Management:** Create, manage, and finalize scouting day events
• **Test Tracking:** Track six different test types with detailed scoring
• **Participant Management:** Manage participant status, grades, and comments
• **Offline Support:** Full functionality without internet connection
• **Cloud Sync:** Automatic synchronization with Firebase when online
• **Responsive Design:** Optimized for both mobile and tablet devices
• **Theme Support:** Light and dark mode options
• **Accessibility:** Adjustable font sizes and UI scaling

3. Technology Stack

3.1 Framework

Flutter: ^3.6.1
Flutter is Google's UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase. The app uses Flutter 3.6.1+ with Dart SDK.

3.2 State Management

GetX 4.6.6: A powerful state management, dependency injection, and route management solution. GetX provides reactive programming with minimal boilerplate, making it ideal for Flutter applications. The app uses GetX controllers for business logic and reactive state management.

3.3 Backend Services

Firebase Core 3.10.1: Firebase initialization and configuration
Cloud Firestore 5.6.2: NoSQL document database for storing events, participants, and grades. Supports offline persistence and real-time synchronization.
Firebase Storage 12.4.1: Cloud storage for media files and documents
Firebase Vertex AI 1.2.0: AI-powered features for participant analysis

3.4 Local Storage

Hive 2.2.3: Lightweight, fast key-value database for local data persistence. Used for caching system settings, user preferences, and offline data storage.
Hive Flutter 1.1.0: Flutter bindings for Hive
Path Provider 2.1.4: Provides platform-specific paths for storing local data

3.5 UI Components

PlutoGrid 8.0.0: Advanced data grid widget for displaying and editing tabular data. Used in the grades page for displaying participant grades.
fl_chart 0.70.2: Feature-rich charting library for creating beautiful charts and graphs. Used for displaying performance analytics and grade distributions.
WebView Flutter 4.13.0: WebView widget for displaying HTML content, used for user guides and documentation.

3.6 Other Dependencies

connectivity_plus 6.1.2: Network connectivity status monitoring
camera 0.10.5+9: Camera functionality for capturing images
image_picker 1.0.6: Image selection from gallery or camera
wakelock_plus 1.2.9: Prevents screen from turning off during tests
intl 0.19.0: Internationalization and localization support
flutter_localization 0.2.3: Additional localization features

3.7 Build Tools

build_runner 2.4.15: Code generation tool for Hive adapters and other generated code
flutter_launcher_icons 0.14.3: Generates app launcher icons
flutter_native_splash 2.3.10: Generates native splash screens

4. Project Structure

4.1 Directory Overview

The project follows a well-organized structure with clear separation of concerns:

```
lib/
■■■ hive
■■■ models
■   ■■■ admin_event.dart
■   ■■■ alonka_sprint.dart
■   ■■■ bur.dart
■   ■■■ display_settings.dart
■   ■■■ display_settings.g.dart
■   ■■■ event.dart
■   ■■■ grade_settings.dart
■   ■■■ instructor.dart
■   ■■■ instructor.g.dart
■   ■■■ meshulash_round.dart
■   ■■■ misc.dart
■   ■■■ participant.dart
■   ■■■ sakim_round.dart
■   ■■■ system.dart
■   ■■■ system.g.dart
■   ■■■ system_settings.dart
■   ■■■ types.dart
■■■ pages
■   ■■■ alonka_page.dart
■   ■■■ bur_page.dart
■   ■■■ event_settings_page.dart
■   ■■■ grades_page.dart
■   ■■■ interview_page.dart
■   ■■■ leadership_page.dart
■   ■■■ login_page.dart
■   ■■■ meshulash_page.dart
■   ■■■ participants_status_page.dart
■   ■■■ performance_page.dart
■   ■■■ sakim_page.dart
■   ■■■ splash_screen.dart
■■■ utils
■   ■■■ logger.dart
■   ■■■ tablet_utils.dart
■■■ widgets
■   ■■■ alonka_charts.dart
■   ■■■ alonka_credit_dialog.dart
■   ■■■ alonka_credit_panel.dart
■   ■■■ alonka_round_panel.dart
■   ■■■ alonka_sprint_panel.dart
■   ■■■ animated_wheels.dart
■   ■■■ bur_charts.dart
■   ■■■ bur_grade_panel.dart
■   ■■■ comments_dialog.dart
■   ■■■ event_grades_distribution_charts.dart
■   ■■■ guideWebView.dart
■   ■■■ interview_chart.dart
■   ■■■ last_update_widget.dart
■   ■■■ leadership_chart.dart
```

4.2 Key Directories

lib/models/

Data models representing the application's domain entities:
• event.dart - Event data structure
• participant.dart - Participant information and grades
• alonka_sprint.dart, sakim_round.dart, meshulash_round.dart - Test-specific models
• bur.dart - Bur test model
• instructor.dart - Instructor information
• grade_settings.dart - Grade calculation settings
• types.dart - Enumerations and type definitions

lib/pages/

UI pages for different app screens (15+ pages):
• login_page.dart, splash_screen.dart - Authentication
• home_page.dart - Instructor home dashboard

• event_home.dart - Event management hub
• alonka_page.dart, sakim_page.dart, meshulash_page.dart, bur_page.dart - Test pages
• leadership_page.dart, interview_page.dart - Additional test pages
• grades_page.dart - Grades overview and editing
• participants_status_page.dart - Participant status management
• performance_page.dart - Individual participant performance
• event_settings_page.dart - Event configuration

lib/widgets/

Reusable UI components (27+ widgets):
• Test-specific widgets: alonka_round_panel.dart, sakim_round_panel.dart, etc.
• Chart widgets: alonka_charts.dart, bur_charts.dart, meshulash_charts.dart
• UI components: strobe_button.dart, comments_dialog.dart, guideWebView.dart
• Grid views: meshulash_grid_view.dart, sakim_grid_view.dart

lib/utils/

Utility functions and helpers:
• logger.dart - Custom logging utility for debugging
• tablet_utils.dart - Tablet detection and responsive utilities

Controllers

Main controllers in lib/ root:
• event_controller.dart - Core event and participant management
• theme_controller.dart - Theme and UI preferences
• connectivity_controller.dart - Network status monitoring

5. Architecture

5.1 Design Pattern

The application follows an **MVC-like architecture** with GetX controllers:
• **Models:** Data structures in lib/models/ representing domain entities
• **Views:** UI pages and widgets in lib/pages/ and lib/widgets/
• **Controllers:** Business logic and state management in GetX controllers

5.2 State Management

GetX provides reactive state management using Rx variables:
• Observable variables (Rx) automatically update UI when values change
• Controllers extend GetxController and manage related state
• Obx widgets rebuild automatically when observed values change
• Dependency injection through Get.put() and Get.find()

5.3 Data Flow

Cloud to Local:
1. Firebase Firestore stores events and participant data
2. Firestore offline persistence caches data locally
3. Hive provides additional local storage for settings and preferences
Local to Cloud:
1. User actions update local state in controllers
2. Changes are saved to Firestore (with offline queue if offline)
3. Firestore automatically syncs when connection is restored

5.4 Route Management

GetX navigation provides declarative route management:
• Routes defined in main.dart using GetPage
• Navigation via Get.toNamed() and Get.offNamed()
• Route parameters passed through URL-like syntax
• Custom transitions and route guards supported

5.5 Offline-First Architecture

The app is designed to work offline-first:
• Firestore enables offline persistence by default
• All read/write operations work without internet
• Changes are queued and synced when connection is restored
• Hive provides additional local caching for critical data
• Connectivity controller monitors network status

6. Key Features

6.1 Test Types

The app tracks six different test types:
• **Meshulash (Triangle):** Team coordination and problem-solving test
• **Alonka (Stretcher):** Physical endurance and teamwork test
• **Bur (Pit):** Obstacle course and physical challenge
• **Sakim (Bags):** Physical strength and endurance test
• **Leadership:** Leadership assessment and evaluation
• **Interview:** Personal interview and evaluation

6.2 Grade Management

Comprehensive grade tracking and calculation:
• Individual test grades for each participant
• System-calculated grades based on test performance
• Instructor override grades
• Final grade calculation with configurable weights
• Grade distribution charts and analytics

6.3 Participant Management

Full participant lifecycle management:
• Participant status tracking (Active, Inactive, etc.)
• Drag-and-drop status changes
• Individual performance pages with detailed analytics
• Comment system for each test type
• AI-powered participant reports

6.4 Responsive Design

Optimized for multiple device types:
• Mobile-first design with tablet optimizations
• Responsive grid layouts (3 columns mobile, 4 columns tablet)
• Scaled fonts and UI elements for tablets
• Touch-optimized controls and gestures
• Portrait-only orientation

6.5 Theme and Accessibility

User customization options:
• Light and dark theme support
• Adjustable font sizes (Normal, Big, Biggest)
• Customizable UI element sizes

7. Development Setup

7.1 Prerequisites

• Flutter SDK 3.6.1 or higher
• Dart SDK
• Android Studio / Xcode (for mobile development)
• Firebase project setup
• Git for version control

7.2 Installation

1. Clone the repository
2. Run 'flutter pub get' to install dependencies
3. Configure Firebase (place google-services.json in android/app/)
4. Run 'flutter run' to start the app

7.3 Platform Support

• **Android:** Minimum SDK 23, Target SDK 36
• **Web:** Supported (requires separate Firebase web configuration)
• **iOS:** Not currently configured

8. Appendix

8.1 Complete Dependencies List

Package	Version	Category
get	^4.6.6	State Management
cloud_firestore	^5.6.2	Firebase
firebase_core	^3.10.1	Firebase
firebase_storage	^12.4.1	Firebase
firebase_vertexai	^1.2.0	Firebase
hive	^2.2.3	Local Storage
hive_flutter	^1.1.0	Local Storage
hive_generator	^2.0.1	Local Storage
pluto_grid	^8.0.0	UI Components
fl_chart	^0.70.2	UI Components
webview_flutter	^4.13.0	UI Components
camera	0.10.5+9	Media
image_picker	1.0.6	Media
path_provider	2.1.4	Utilities
intl	^0.19.0	Utilities
connectivity_plus	^6.1.2	Utilities
wakelock_plus	1.2.9	Utilities
flutter_launcher_icons	^0.14.3	Build Tools
flutter_native_splash	2.3.10	Build Tools
cupertino_icons	^1.0.8	Other
flutter_localization	^0.2.3	Other

8.2 File Count Summary

• Models: 15+ files
• Pages: 15+ files
• Widgets: 27+ files
• Controllers: 4 main controllers
• Total Dart files: 60+